OMNI

Speak anywhere. Act everywhere.

One AI brain. Every device. Infinite capabilities.

Omni is a multi-client AI agent hub that lets you speak to one intelligent agent from any device — web dashboard, mobile, Chrome extension, desktop, or smart glasses — and have it act across all of them simultaneously.

Demo Video · Architecture · Getting Started · Blog Post

The Problem

AI assistants today live in text boxes on single screens. You can't speak to your AI while wearing safety glasses on a factory floor. You can't add new capabilities without waiting for the next software update. You can't switch devices mid-thought and pick up where you left off.

Every AI assistant is an island.

The Solution

Omni connects one AI brain to every device you own. Speak from your phone, see results on your dashboard, trigger actions on your desktop — all in one continuous conversation.

- **One voice, every device** — Web, mobile, Chrome extension, desktop tray app, ESP32 glasses
- **MCP Plugin Store** — Install new agent capabilities in one click, like an app store for AI skills
- **GenUI** — Agent renders live charts, tables, code blocks, and cards on your dashboard while speaking to you
- **Agent Personas** — Switch between specialized AI personalities (analyst, coder, researcher) with distinct voices and skills
- **Browser Control** — Tell your agent to scrape a website, fill a form, or extract data — all by voice
- **Cross-Client Actions** — Say "save this to my dashboard" from your phone → it appears on your desktop instantly

---

Demo

> 🎥 Watch the 4-minute demo video → *(coming soon)*

Highlights

Moment	What Happens
Voice + GenUI	Ask about stock performance → agent speaks the answer while a chart renders live on the dashboard
Persona Switch	"Switch to Atlas" → voice changes instantly → ask for code → code block renders → "Execute it" → runs in sandbox
MCP Plugin Toggle	Enable Brave Search with one click → agent immediately searches the web → disable it → agent falls back gracefully
Cross-Client	Point phone camera at an object → agent describes it → "Saved to your dashboard" → switch to desktop → it's there

---

Architecture

```
                    ┌──────────────────────┐
                    │      OMNI HUB        │
                    │   (Cloud Run)        │
                    │                      │
                    │  ┌────────────────┐  │
                    │  │  Root Agent    │  │
                    │  │  (ADK)         │  │
                    │  │                │  │
                    │  │  ┌─────┐ ┌─────┐  │
                    │  │  │Nova │ │Atlas│  │
                    │  │  │     │ │     │  │
                    │  │  └─────┘ └─────┘  │
                    │  │  ┌─────┐ ┌─────┐  │
                    │  │  │Sage │ │Spark│  │
                    │  │  └─────┘ └─────┘  │
                    │  └────────┬──────────┘
                    │           │
                    │  ┌────────┴──────────┐
                    │  │ MCP Plugin System │
                    │  │ (Dynamic Tools)   │
                    │  └────────────────┘  │
                    └──────────┬──────────┘
                               │
          ┌────────────────────┴────────────────┐
          │     Raw WebSocket (Binary           │
          │     audio + JSON control)           │
          │                                     │
┌─────────┴──┐   ┌──────────┴──┐   ┌──────────┴──┐
│ Web        │   │ Mobile      │   │ Chrome      │
│ Dashboard  │   │ PWA         │   │ Extension   │
│ (React)    │   │ (Camera)    │   │ (Voice)     │
└────────────┘   └─────────────┘   └─────────────┘

┌────────────┐   ┌────────────┐
│ Desktop    │   │ ESP32      │
│ Tray App   │   │ Glasses    │
│ (Python)   │   │ (Protocol) │
└────────────┘   └────────────┘
```

Key Design Decisions

Decision	Choice	Why
Audio Transport	Binary WebSocket frames	33% smaller than base64-in-JSON, lower latency
Audio Pipeline	AudioWorklet (not ScriptProcessor)	Runs on separate thread, zero main-thread jank
Agent Framework	Google ADK with `run_live()`	Native bidi streaming, multi-agent orchestration
Plugin System	MCP (Model Context Protocol)	Open standard, 10,000+ community tools
GenUI	Agent returns structured JSON → React renders	Audio + visual output simultaneously
Session Persistence	Vertex AI Agent Engine Sessions	Survives Cloud Run restarts, Google-managed

---

Tech Stack

Backend

Component	Technology
Runtime	Python 3.12+
Package Manager	uv
API Server	FastAPI + Uvicorn
Agent Framework	Google ADK v0.5+
Audio Model	`gemini-live-2.5-flash-native-audio`
Code Execution	E2B Sandbox + Agent Engine Code Execution

Frontend

Component	Technology
Framework	React 19 (JavaScript)
Build Tool	Vite
Styling	Tailwind CSS 4 + shadcn/ui
State	Zustand
Charts	Recharts
Icons	Lucide React
Toasts	Sonner

Google Cloud Services (16+)

Category	Services
Vertex AI	Gemini Live API, ADK, Grounding (Google Search + Maps), Agent Engine (Sessions + Memory Bank + Code Execution), Gen AI Evaluation, Imagen 4
Infrastructure	Cloud Run, Firestore, Firebase Auth, Cloud Storage, Secret Manager, Artifact Registry
Observability	Cloud Logging, Cloud Monitoring, Cloud Trace
DevOps	Cloud Build, Terraform

Getting Started

Prerequisites

- Python 3.12+
- Node.js 20+
- uv (Python package manager)
- Google Cloud account with billing enabled
- Google Cloud CLI (`gcloud`)

1. Clone the repository

```
git clone https://github.com/omanandswami2005/omni-agent-hub-with-gemini-live.git
cd omni-agent-hub-with-gemini-live
```

2. Set up environment variables

```
cp .env.example .env
# Edit .env with your API keys:
#   GOOGLE_CLOUD_PROJECT=your-project-id
#   GOOGLE_CLOUD_LOCATION=us-central1
#   E2B_API_KEY=your-e2b-key
```

3. Start the backend

```
cd backend
uv sync
uv run uvicorn main:app --reload --port 8000
```

4. Start the frontend

```
cd frontend
npm install
npm run dev
```

5. Open the dashboard

Navigate to `http://localhost:5173` — click “Sign in with Google” and start talking.

Deploy to Google Cloud

One-command deploy

```
cd deploy
terraform init
terraform apply
```

Manual deploy

```
# Build and push container
gcloud builds submit --tag gcr.io/$PROJECT_ID/omni-backend

# Deploy to Cloud Run
gcloud run deploy omni-backend \
  --image gcr.io/$PROJECT_ID/omni-backend \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --min-instances 1 \
  --set-env-vars "GOOGLE_CLOUD_PROJECT=$PROJECT_ID"
```

Agent Personas

Omni ships with 5 specialized AI personas, each with a unique voice, personality, and toolset:

Persona	Role	Voice	Specialization
Nova	Analyst	Charon	Financial analysis, data interpretation, market research
Atlas	Coder	Kore	Code generation, debugging, architecture, E2B execution
Sage	Researcher	Aoede	Deep research, fact-checking, academic analysis
Spark	Creative	Fenrir	Writing, brainstorming, content creation, image generation
Claire	Assistant	Leda	General tasks, scheduling, email, daily planning

Switch personas by voice: *"Switch to Atlas"* — or click the persona panel on the dashboard.

Create custom personas: *"Create a new persona called Chef with a warm voice and cooking knowledge."*

MCP Plugin Store

Omni's agent capabilities are extensible at runtime through the MCP Plugin Store:

Plugin	What It Does	Source
Brave Search	Web search via Brave API	Community MCP
Google Maps	Location data and directions	Built-in (Grounding)
GitHub	Repo management, issues, PRs	Community MCP
Slack	Send/read messages	Community MCP
Wikipedia	Encyclopedia lookups	Community MCP
Context7	Up-to-date code documentation	Community MCP
Filesystem	Read/write local files	Community MCP
Chrome DevTools	Browser automation and scraping	Community MCP
E2B Sandbox	Code execution (100+ languages)	E2B Gateway
Memory	Long-term memory storage	Agent Engine

Enable/disable any plugin with a single toggle — the agent adapts instantly. No restart required.

Project Structure

```
omni-agent-hub-with-gemini-live/
├── backend/                    # Python FastAPI + ADK
│   ├── agents/                 # Agent definitions
│   │   ├── root_agent.py       # Root orchestrator
│   │   ├── personas/           # Nova, Atlas, Sage, Spark, Claire
│   │   └── task_architect.py   # Meta-orchestrator (CustomAgent)
│   ├── mcp/                    # MCP plugin management
│   ├── tools/                  # Custom ADK tools
│   ├── services/               # Session, auth, storage services
│   ├── websocket/              # WebSocket handler (binary audio + JSON)
│   └── main.py                 # FastAPI app entry point
├── frontend/                   # React 19 + Vite
│   ├── src/
│   │   ├── components/         # UI components (shadcn/ui)
│   │   ├── pages/              # Dashboard, Personas, Plugins, Sessions...
│   │   ├── stores/             # Zustand state stores
│   │   ├── hooks/              # useWebSocket, useAudioPipeline, etc.
│   │   ├── audio/              # AudioWorklet recorder + streamer
│   │   └── genui/              # Dynamic GenUI component renderer
│   └── index.html
├── clients/
│   ├── chrome-extension/       # Manifest V3 + vanilla JS
│   ├── desktop/                # Python + pystray + pyautogui
│   └── esp32/                  # Arduino WebSocket client (protocol only)
├── deploy/
│   ├── terraform/              # Cloud Run + Firestore + GCS + Secret Manager
│   ├── Dockerfile
│   └── scripts/
├── docs/                       # Architecture diagrams, API docs
├── .env.example
└── README.md
```

How It Works

Voice Pipeline

```
User speaks → Mic → AudioWorklet (16kHz PCM)
  → Binary WebSocket frame → FastAPI backend
    → ADK LiveRequestQueue → Gemini Live API
      → Agent processes (tools, grounding, GenUI)
    ← ADK response (audio + text + structured data)
  ← Binary frame (24kHz PCM) + JSON frames (transcript, GenUI, status)
← AudioWorklet playback → Speaker

Latency target: < 500ms end-to-end
```

Cross-Client Actions

```
Phone camera → captures image → sends via WebSocket
  → Agent: "This is a book about machine learning"
  → Agent: "I've saved the analysis to your dashboard"
  → WebSocket push to dashboard client
    → GenUI card appears with image + analysis

All clients share the same session via Agent Engine Sessions
```

GenUI Flow

```
User: "Show me Tesla's stock performance"

Agent response:
  Audio: "Tesla has been on an upward trend..."
  GenUI: {
    type: "chart",
    chartType: "line",
    data: [...],
    title: "Tesla (TSLA) — 12 Month Performance"
  }

Dashboard renders Recharts component inline in chat
while audio plays simultaneously
```

Judging Criteria Alignment

Criterion	Weight	Our Score Target	Key Features
Innovation & Multimodal UX	40%	5/5	Multi-client hub, GenUI, cross-client actions, voice personas, MCP plugin store, browser control
Technical Implementation	30%	5/5	14 ADK features, 16+ GCP services, Agent Engine, binary audio transport, AudioWorklet pipeline
Demo & Presentation	30%	5/5	4-min scripted video with "wow" moments, architecture diagram, Cloud deployment proof
Bonus	+1.0	+1.0	Blog post (+0.6), Terraform deploy (+0.2), GDG membership (+0.2)

Blog Post

> 📝 How I Built a Multi-Device AI Agent Hub with Gemini Live API & Google ADK → *(coming soon)*

Team

Name	Role
Your Name	Full-stack developer

License

This project is built for the Gemini Live Agent Challenge hackathon.

OMNI — Speak anywhere. Act everywhere.

Built with ❤️ using Google Gemini, ADK, and 16+ Google Cloud services.